idox mobile: 2B model (Qwen3-VL-2B-Instruct)

On-device PDF-to-Word conversion · Samsung Galaxy M35 (SM-M356B, Exynos 1380) · CPU only · as of 2026-10-12

1. Summary

Document	First runs	Now	Accuracy now
Digital doc (2 pages)	245–354 s (image mode)	**71.6 s**	**100/100**
Test doc (2 pages)	730 s (early text mode, loops)	**95.3 s**	**100/100**
Scanned doc (2 pages)	345.4 s	**199.3 s**	≈93/100 (footer missed)
Peak memory	2.46–2.57 GB	2.07–2.32 GB (old libs) / 2.66 GB (new libs)	

2. Configuration

Setting	Value
Model files	`qwen3vl-2b-instruct-q4_k_m.gguf` + projector `mmproj-qwen3vl-2b-instruct-q8_0.gguf` (445 MB)
Runtime	llama.cpp `llama-server` bundled as `libllama-server.so`, started from `nativeLibraryDir` with `LD_LIBRARY_PATH`
Server options	`-t 4 -tb 4 -Cr 4-7 -Crb 4-7 --cpu-strict 1 --cpu-strict-batch 1 -fa on -np 1 -c 8192 -ctk q8_0 -ctv q8_0 -ngl 0 --no-mmproj-offload --no-op-offload` (no `--jinja`)
Image tokens	Fixed at 512 via `LLAMA_ARG_IMAGE_MIN_TOKENS` / `LLAMA_ARG_IMAGE_MAX_TOKENS`
KV cache	**8-bit (q8_0)**: 112 → 59.5 KiB per token, about −410 MB peak, accuracy unchanged. 4-bit rejected (accuracy ≈84)
llama.cpp build	Commit `000bee54a544070356b44d68af299b973d391d26`, NDK 27.1.12297006, CMake 3.22.1, `-DGGML_NATIVE=OFF -DGGML_CPU_ARM_ARCH=armv8.2-a+dotprod+fp16 -DGGML_OPENMP=ON -DGGML_VULKAN=OFF -DGGML_OPENCL=OFF`; libraries in `jniLibs/arm64-v8a` plus NDK `libomp.so`
Request path	`/apply-template` then streamed `/completion` with `json_schema` or grammar (`continue_blocks.gbnf` for continuation)
Decoding	temperature 0; `repeat_penalty` 1.0 text mode, 1.15 image mode

3. How a page is processed

- **Digital page (text mode):** PdfBox-Android reads the text layer as Markdown (headings, | tables, captions). The model structures it into blocks. Continue only if text-layer words are missing. `TextRepair` then fixes structure against the text layer.
- **Scanned page (image mode):** the scan image stored in the PDF is sent directly, no re-rendering. JPEG passed as-is; other formats decoded from the stored samples. Blank-image guard; rendering only as last resort. Continue when the model's answer stops early.
- Every page: `PageGuard` loop stop, salvage, dedupe, Markdown clean-up; `DocxWriter` writes the Word file.

4. Problems faced and how they were solved

#	Problem	Fix
1	Bottom UI row hidden (edge-to-edge)	Window insets + NoActionBar theme
2	adb-copied models unreadable by the app	Copied with `run-as` into private storage

3	`/v1/chat/completions` HTTP 500 (peg-native)	`/apply-template` + `/completion`
4	Low-memory kills	Foreground service `ConvertService`; 8-bit KV cache (−410 MB)
5	Prompt "promise" rule caused loops; relaxing it caused early stops	Continuation from the model's own state instead of a retry
6	A table re-written until the token limit; text-only loop check could not see it	Value-run loop check (6 values repeated 3×). A tokenising bug gave false loops; fixed
7	Model re-wrote page 1 from the top after finishing	"Done" stop: all words written and a block repeated, counting completed blocks only (an earlier version cut a block mid-way)
8	Chart rule produced junk	Rule 7 rewritten (chart = image block)
9	Structure errors (folded headings, rows outside tables, wrong order)	`TextRepair`; two bugs fixed (moved row kept only first line; duplicate heading joined instead of dropped)
10	Scanned page came out blank: PdfBox put gray values into alpha	Own decoder `samplesToPng()` for gray/RGB 8-bit and 1-bit samples (honours inverting Decode) + blank guard
11	Test runs invalid after adb disconnect / cached image	Re-ran with a guard that rejects cached runs

5. Latency: how it was reduced

Step	Measure	Effect
1	**Text mode** for digital pages (text layer instead of image)	Digital doc 245–354 s → 170–180 s
2	Block limit, "done" stop, loop stop, continuation only when words are missing	Test doc 730 s → 126–132 s; digital doc → 127.4 s
3	8-bit KV cache	Peak memory 2.48 → 2.07 GB, same accuracy
4	Scan: stored image sent directly (no redraw) + fixes above	Scan 345.4 s → 288.5 s
5	**Dot-product build** of llama.cpp (ARMv8.2 SDOT + FP16; 1,139 SDOT instructions vs 0; `DOTPROD = 1` confirmed)	Image reading 164.5 → 79.6 s (2.07×). Digital −44%, test −28%, scan −31%. Writing unchanged (~9.1 tok/s)

Tried and rejected

Experiment	Result
4-bit KV cache	Accuracy ≈84; rejected
Crop blank paper: separate band images	Model stopped after the title
Crop: stitched bands, same detail	−30% time, mixed accuracy (page 1 worse); undecided, needs repeat runs
Crop: lower detail	Fails
OpenMP thread pinning	No gain on old or new build
Vulkan GPU	Removed in the dot-product build; GPU checkbox no longer works

6. Results on current build (dot-product libraries, 2026-10-12)

Document	Page 1	Page 2	Total	Old libs	Words	Score
Digital doc	32.1 s	39.5 s	**71.6 s**	127.4 s	119/119	**100**
Test doc	43.0 s	52.4 s	**95.3 s**	131.9 s	118/118	**100**
Scanned doc	94.3 s	105.0 s	**199.3 s**	288.5 s	131/140	≈93

Scan: reading 45.1 + 36.2 s (was 87.3 + 66.4 s); writing 48.8 + 68.3 s. Numbers 6/6, table 12/12. Misses: footer on both pages (every scan run), and the "Example Data Table" heading on page 2 (new, possibly run-to-run variation).

7. Open items

- Peak memory rose to 2.66 GB on the new build; likely the RAM prompt cache (default 8192 MB). Try `--cache-ram 512`.
- Repeat scan runs to check the missing heading; footer still missed.
- Empty image blocks in image mode; stitched-crop repeat runs; warm-up request decision.